

Kairos: A Peer-to-Peer Electronic Cash System Built to Last

The Kairos developers

Drafted with Claude, an AI model made by Anthropic

Reference implementation: kairos-0.3.0 (revision 3)

September 23, 2026

Abstract.

Bitcoin solved double-spending without a trusted party, and seventeen years of operation have confirmed that the core design is sound. The same years have also exposed where it strains: a security budget that is scheduled to shrink toward zero, fee markets that swing violently, public keys that a large quantum computer could break, a full-history download that grows without bound, and consensus quirks that had to be patched after launch. Kairos keeps what worked unchanged, including proof-of-work over SHA-256d, the UTXO model, and longest-accumulated-work consensus, and redesigns the parts that time revealed. Issuance decays smoothly into a small perpetual tail, and a burned base fee lets usage offset it. Every block header commits to the entire UTXO set with an incremental multiset hash, so new nodes can start from a recent state verified by proof-of-work. Every address commits to both an elliptic-curve key and a hash-based one-time key, so that if elliptic-curve cryptography falls, coins can move under a pre-committed post-quantum key rather than being stolen or frozen. Difficulty adjusts every block, and Bitcoin miners can secure the chain through merge-mining from its first day. Malleability, Merkle ambiguity, duplicate coinbases and cross-chain replay are excluded by construction.

1. Introduction

The 2008 design asked participants to trust only mathematics and the honest majority of CPU power. That promise has held. What a successor must address is not whether the system works but whether it keeps working as conditions change over decades: as block subsidies approach zero, as the chain grows past what ordinary machines can replay, and as the cryptographic assumptions underneath age.

We take a conservative position. A component is changed only when operating experience has shown a concrete failure mode and a well-understood remedy exists. Every mechanism in Kairos has either been deployed elsewhere or analysed in the literature; the contribution is in choosing them together, fixing them into consensus from the first block, and showing that they compose. A new system can do what an existing one cannot: adopt all of these at genesis without the compromises that backward compatibility forces on soft forks.

2. Transactions

As in Bitcoin, a coin is a chain of signatures, and a transaction spends previous outputs and creates new ones. Kairos changes what an output pays to and what a signature signs.

Dual-key addresses. An output pays to a 32-byte commitment to two public keys: a BIP340 Schnorr key [2] used for everyday spending, and the root hash of a Lamport one-time key [6] held in reserve. Neither key is revealed until the output is spent. Schnorr signatures are 64 bytes, provably secure in the random-oracle model, and support key aggregation, so a multi-party output is indistinguishable from a single-party one.

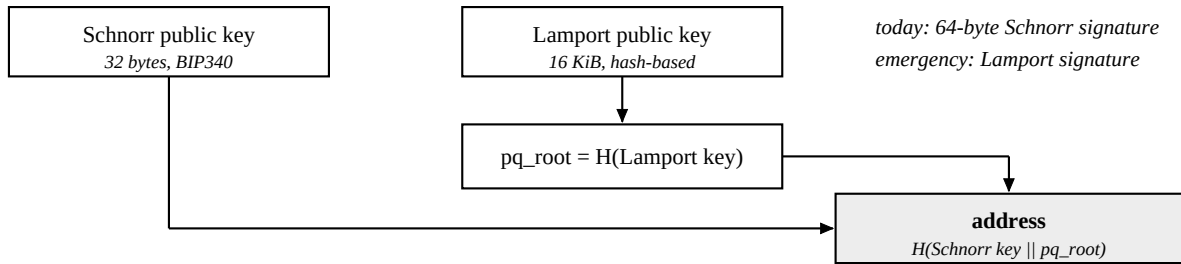


Figure 1. A Kairos address commits to a classical and a post-quantum key.

Witnesses outside the identifier. A transaction's identifier is the hash of its body only: inputs, outputs, version and expiry. Signatures travel in a separate witness section, so no third party can change a transaction's identifier by re-encoding a signature. Bitcoin reached this property in 2017 through BIP141 [8]; Kairos starts with it.

What is signed. The signature hash covers the whole transaction body together with a chain identifier. A transaction signed for one network is invalid on any fork or test network that uses a different identifier, which removes cross-chain replay without special-case rules.

Expiry. A transaction may name a last height at which it can be mined. Payments that fail to confirm in time become permanently invalid instead of lingering, which gives wallets a clean answer to the question of whether an unconfirmed payment can still happen.

3. Timestamp Server and Proof-of-Work

The proof-of-work function is unchanged: a block is valid when the double SHA-256 hash of its header is at most a target. What the first revision of this paper overlooked is what that choice means for a young chain. Sharing Bitcoin's algorithm puts Kairos beside the largest pool of SHA-256 hardware in existence, and anyone who rents a small fraction of it can outpace a new network. Several small SHA-256 chains have been rewritten this way.

Kairos therefore accepts two forms of work. A block may carry its own proof, or an auxiliary proof-of-work: a Bitcoin block header whose coinbase transaction commits to the Kairos block hash, together with the Merkle branch linking that coinbase to the header [11]. The same hashing secures both chains, so Bitcoin miners can protect Kairos at almost no extra cost, and the commitment sits in a Merkle tree of chain slots so one pool can merge-mine several chains at once. A deterministic slot rule stops a miner from placing two Kairos blocks in one tree, and parent coinbases of 64 bytes or less are rejected so that an inner Merkle node cannot pose as a transaction. Merge-mining is not a complete defence: a pool that merge-mines can also attack at near-zero marginal cost, so security grows with the number of independent pools that take part.

The header grows from 80 to 124 bytes to carry three additions. The block height is explicit, so light clients know their position without trusting anyone and no two coinbase transactions can ever share an identifier. The transaction root commits to full witness-inclusive hashes, so signatures are covered directly by proof-of-work. The UTXO root commits to the entire set of unspent outputs after the block (section 7). Timestamps are 64-bit.

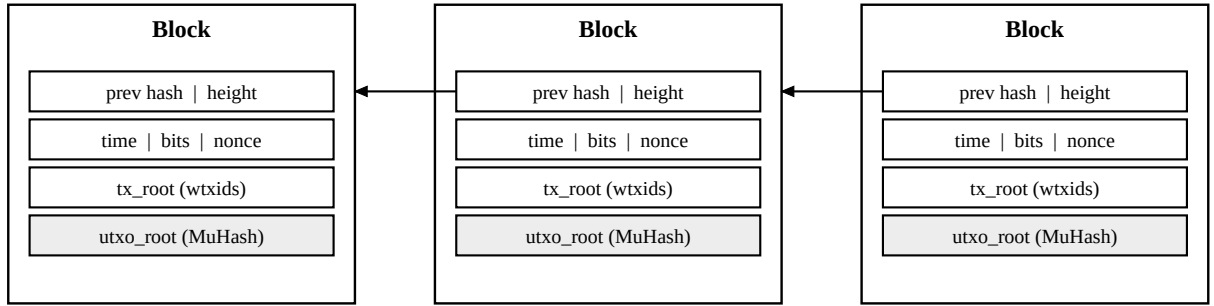


Figure 2. Each header commits to its parent, its transactions, and the complete resulting UTXO set.

4. Difficulty

Bitcoin retargets once every 2016 blocks, which produces oscillation when hash power moves quickly and has enabled timestamp-manipulation attacks at window boundaries. Kairos recomputes the target at every block using an absolutely scheduled exponential rule (ASERT) [3]. With genesis as the anchor, a block's target depends only on how far its parent is ahead of or behind the ideal schedule:

$$target = anchor_target \cdot 2^{(t_{parent} - t_{anchor} - T \cdot h_{parent}) / \tau}$$

where T is the 120-second target spacing and τ is a two-day half-life. Being two days behind schedule doubles the target; two days ahead halves it. The computation uses integer fixed-point arithmetic with a cubic approximation of 2^x , so every implementation agrees to the bit. Timestamps must exceed the median of the previous eleven blocks and may not run more than two hours into the future. The first block after genesis starts at Bitcoin's 2009 minimum difficulty, so the earliest coins cost real work rather than being claimable in a burst.

5. Network

The network runs the same steps as the original design. New transactions are broadcast to all nodes; each node collects them into a block and works on a proof-of-work; when a node finds one it broadcasts the block; nodes accept it only if every transaction in it is valid and not already spent; and they express acceptance by building on its hash. Nodes always consider the chain with the most accumulated work to be correct, and on a tie keep the first they received.

When a heavier branch appears, a node disconnects blocks back to the fork point using stored undo data, connects the new branch, and returns any transactions from abandoned blocks to its memory pool. A block is validated entirely against a copy-on-write view of the UTXO set before any state changes, so an invalid block can never leave a node half-updated.

6. Incentive

Emission without cliffs. Instead of halving every four years, each block's subsidy is a fixed fraction of the coins not yet issued from a 21 million main curve, with a floor:

$$subsidy = \max(0.6 \text{ KRS}, (21,000,000 \text{ KRS} - issued) / 2^{21})$$

The first block pays 10.01 KRS, which at one block per two minutes is 300 coins per hour, the same hourly issuance as Bitcoin in 2009. Half of the main curve is issued after about 5.5 years. There are no sudden drops in miner revenue, so there is no scheduled moment when a large share of hash power

becomes unprofitable at once. After about 22 years the curve reaches the floor and the chain pays 0.6 KRS per block forever: 157,788 KRS per year, about 0.8% of supply at that point and a shrinking fraction thereafter.

Why a tail. A proof-of-work chain is only as secure as what it pays for work. If that payment must eventually come entirely from fees, security becomes as volatile as fee demand, and an empty mempool invites miners to rewrite recent blocks to steal the fees in them. Budish [10] shows the cost of attack is bounded by the flow payment to miners; a permanent floor keeps that bound from collapsing. Monero has operated with a tail emission since 2022 [9].

Burned base fee. Every transaction pays at least a per-byte base fee, which is destroyed, plus an optional tip to the miner. The base fee rises by up to 12.5% after a block larger than the 1 MB target and falls after a smaller one, as in EIP-1559 [4]. Users face a predictable price instead of a blind auction. Because miners cannot collect the base fee, they gain nothing by padding blocks with their own transactions to manipulate it. The burn also closes the monetary loop: net supply growth is the tail minus the burn, and it reaches zero when the average block burns 0.6 KRS, for example full-target blocks at a base fee of 60 motes per byte. Heavy use makes the currency scarcer; light use leaves a small, predictable inflation that pays for security.

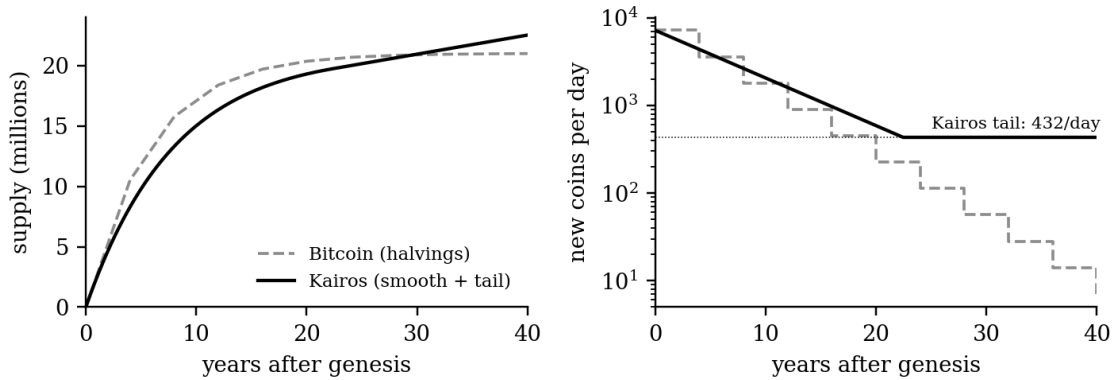


Figure 3. Gross issuance of Kairos compared with Bitcoin. Right: coins created per day, log scale.

7. Committed State and Fast Synchronization

A new Bitcoin node must download and replay every transaction since 2009 to learn the current set of unspent outputs, which is the only state it needs. Kairos commits that state in every header. The commitment is a MuHash over the multiset of unspent outputs [5]: each output is hashed to an element of the multiplicative group modulo the prime $2^{3072} - 1103717$, the set's hash is the product of its elements, adding an output multiplies it in and spending one divides it out. Updates cost constant time per output and are independent of order, so any node can maintain the commitment as it validates.

A new node can therefore download headers, choose a block buried under enough work, fetch a UTXO snapshot from any untrusted peer, and check the snapshot's hash against the header. A false snapshot can only be accepted if a majority of hash power has repeatedly committed to a false UTXO root, which every honest validating node rejects, so this is the same assumption the system already rests on. The node can then validate history in the background for full assurance.

8. Simplified Payment Verification

As in the original design, a light client keeps only headers and checks that a transaction is included by its Merkle branch. Kairos uses different hash tags for leaves and inner nodes and promotes an odd node instead of duplicating it. This removes two known Bitcoin weaknesses: the duplicated-leaf ambiguity that allowed CVE-2012-2459 [7], and the possibility of a 64-byte transaction being mistaken for an inner node in a Merkle proof. Light clients can also check balances against the committed UTXO root once proofs of set membership are added.

9. Surviving a Quantum Adversary

A sufficiently large quantum computer running Shor's algorithm would recover the private key for any exposed elliptic-curve public key. Hash functions are only weakened by Grover's algorithm, to roughly 128-bit security for SHA-256. Kairos prepares for this at genesis instead of at a moment of crisis.

Every address already commits to a Lamport key whose security rests only on SHA-256. Consensus includes an emergency height, unset at launch, which the network can activate by soft fork if elliptic-curve cryptography is broken in practice. From that height, Schnorr spends are invalid and every output can still be spent with its Lamport key. Nobody's coins are frozen and none are stolen: an attacker who derives a Schnorr private key gains nothing, because that key no longer authorises anything.

A Lamport key must sign only one message. The signature hash covers the whole transaction, so every input from one address signs the identical message; the reference wallet therefore sweeps every coin at an address in a single transaction and sends change to a fresh, never-revealed key. Lamport witnesses are about 24 KB, so emergency transactions are large and expensive. That is the right trade: this is an escape path, and more compact hash-based schemes such as SPHINCS+ can replace it by soft fork later, since only the committed root is fixed.

10. Calculations

The probability that an attacker with fraction q of the hash power ever catches up from z blocks behind follows the gambler's-ruin analysis of the original paper [1] and is unchanged, since it depends on blocks rather than seconds. What changes is wall-clock time. The table gives the confirmations needed to bring the attacker's success probability below 0.1%.

q	blocks z	Kairos (2 min)	10-minute blocks
0.10	5	10 min	50 min
0.15	8	16 min	80 min
0.20	11	22 min	110 min
0.25	15	30 min	150 min
0.30	24	48 min	240 min
0.35	41	82 min	410 min
0.40	89	178 min	890 min

Shorter blocks are not free. With a block propagation time of a few seconds, roughly 2 to 3% of blocks at a two-minute spacing will be orphaned, against a fraction of a percent at ten minutes, which

gives a small advantage to well-connected miners. We judge a five-fold improvement in settlement time to be worth this, but compact block relay is a prerequisite for a production network.

11. Reference Implementation and Limitations

The accompanying kairo-0.3.0 implements everything above in about 3,300 lines of Python: BIP340 Schnorr with a constant-time libsecp256k1 backend and a readable reference fallback, Lamport signatures, MuHash3072, domain-separated Merkle trees, bech32m addresses, merge-mining proofs, full consensus validation with chain reorganisation, a bounded memory pool, crash-safe storage, an encrypted deterministic wallet, a peer-to-peer protocol with misbehaviour scoring, rate limits and bans, automatic reconnection and peer discovery, and a Bitcoin-style JSON-RPC including the calls merge-mining pools use. Its 55 tests include a differential test showing that the two signature backends agree on valid and malformed inputs, fuzzing of blocks and transactions, attacks by misbehaving peers, and a block merge-mined end to end through the RPC interface.

It is a public-testnet candidate, not a system for real value. It has not been independently audited; it keeps all blocks in memory; headers-first synchronisation, compact block relay, soft-fork activation, MuSig2 and UTXO snapshot transfer remain to be built; and compatibility with real merge-mining pool software has yet to be demonstrated. The conditions for a mainnet launch, including independent audits, a second implementation and at least six months of public testnet, are set out in the release's LAUNCH document. None of the remaining work requires changing the consensus rules described here.

12. Conclusion

We have proposed a system for electronic transactions without relying on trust that keeps the proof-of-work foundation of the original and changes only what long operation has shown to need changing. Its security budget has a floor, and usage can offset the inflation that floor creates. Its fees are predictable. Its state is committed so that joining the network does not require replaying history. Its coins have a pre-committed path through the failure of elliptic-curve cryptography. Its consensus rules exclude by construction the malleability, Merkle, coinbase and replay problems that Bitcoin had to patch. The network remains robust in its unstructured simplicity: nodes work all at once with little coordination, need not be identified, and vote with their CPU power on the valid chain.

References

- [1] S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008.
- [2] P. Wuille, J. Nick, T. Ruffing, "BIP340: Schnorr Signatures for secp256k1," 2020.
- [3] M. Lundeberg, J. Toomim et al., "ASERT difficulty adjustment algorithm (aserti3-2d)," Bitcoin Cash upgrade specification, 2020.
- [4] V. Buterin, E. Conner, R. Dudley, M. Slipper, I. Norden, A. Bakhta, "EIP-1559: Fee market change for ETH 1.0 chain," 2019.
- [5] M. Bellare, D. Micciancio, "A New Paradigm for Collision-free Hashing: Incrementality at Reduced Cost," EUROCRYPT 1997.
- [6] L. Lamport, "Constructing Digital Signatures from a One Way Function," SRI International, 1979.
- [7] CVE-2012-2459, Bitcoin Merkle tree duplicate-transaction vulnerability, 2012.
- [8] E. Lombrozo, J. Lau, P. Wuille, "BIP141: Segregated Witness (Consensus layer)," 2015.

- [9] Monero Project, tail emission, active since May 2022.
- [10] E. Budish, "The Economic Limits of Bitcoin and the Blockchain," NBER Working Paper 24717, 2018.
- [11] Namecoin project, merged mining specification (auxiliary proof-of-work), 2011.